Data Science

Deep Learning

Sylvain BENOIT & Arthur THOMAS

Dauphine | PSL
UNIVERSITÉ PARIS

Outline

1. Introduction
2. Deep learning
3. Parameters estimation
4. Supervised learning
5. More complex neural
6. Conclusion

Recommended Textbooks

- Aurélien Géron (2019) Hands-On Machine Learning with Scikit-Learn, Keras & and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems, 2nd Edition.
  $\Rightarrow$ focuses on application with a bit of theory, chapters with Python code and data, links to websites with data and pretrained neural networks
- Goodfellow, Bengio, Courville (2016) Deep Learning (Adaptive Computation and Machine Learning series)
  $\Rightarrow$ widely considered as the "Bible" of Deep Learning, technical.
- François Cholet (2018) Deep Learning with Python, Manning Publications.

Introduction

- artificial neural network: inspiration by function of brains
- inspiration should not be taken too literally; analogy: planes were inspired by birds, but they do not flap their wings
- some researchers want to change terminology from "neurons" to "units" to avoid restricting creativity
- Artificial Neural Networks are at the core of Deep Learning, used for
  - classifying billions of images (e.g. Google Images)
  - speach recognition (Apple's Siri)
  - recommending best videos (e.g. Youtube)
  - beating the world champion in the game of Go (DeepMind's AlphaGo)

History

- McCulloch and Pitts (1943) "A Logical Calculus of Ideas Immanent to Nervous Activity"
- Early 1960s the idea for Artifical Neural Network research was to converse with truly intelligent machines
- Funding was cut when this hope was not fulfilled
- In early 1980s revival of Artificial Neural Networks as new network architectures and better training techniques were developed
- In 1990s focus shifted to other Machine Learning techniques such as *Support Vector Machines*

History

- Currently: another wave of interest in Artificial Neural Networks
- This time, Artificial Neural Networks seem to be here to stay, because
  - Huge quantity of data, Artificial Neural Networks frequently outperform other Machine Learning Techniques on very large and complex problems
  - Tremendous increase in computing power (Moore's Law and cheap Graphical Processing Units due to gaming industry)
  - Training algorithms have improved
  - Some theoretical limitations turned out not to matter in practice (local vs global minima)
  - Virtuous circle of funding and progress

Applications in Economics/Finance

- So far applications mostly in image classification, speech recognition, recommendation systems, sentiment analysis
- Direct application to economic/financial data: feed data to neural network, make prediction $\Rightarrow$ the jury is still out on this
- Use image classification, speech recognition, sentiment analysis as inputs/instruments for economic financial analysis $\Rightarrow$ safe bet
  - Positive/negative articles in newspapers as one variable that explains share prices
  - Positive/negative articles in newspapers as one variable that affects policies/elections
  - Image recognition for satellite images to infer crops being grown (Refinitiv)

What's the difference between estimation and learning?

- **Statistical tradition / estimation:**
  - Concept of model is central with an explanatory purpose.
  - Seeking to approach reality, propose a model possibly based on a physical theory, economic.
  - Interpretation of the role of each explanatory variable in the process is important.

- **Learning: the objective is essentially the prediction,**
  - Best model is not necessarily the one that best fits the true model.
  - Theoretical framework is different and error control requires another approach: choices based on prediction quality criteria.
  - Interpretability is less important.

So what about the good old (generalized) linear model?

$$y_i \sim \mathcal{F} \quad \text{with} \quad \phi(\mathbb{E}[y_i]) = \mathbf{x}^T \beta$$

- If the dimensions of the problem (n, p) are reasonable
- If model assumptions (linearity) and distributions are verified
- THEN: statistical modeling techniques issued from the general linear model are optimal (maximum likelihood)
- We will not do better, especially in the case of small samples

But...

- As soon as the distribution hypotheses are not verified,
- As soon as the supposed relations between the variables or with the variable of interest are not linear
- As soon as the data volume is important (big data),

We will consider other methods that will over-rate rudimentary statistical models.

Outline

1. Introduction
2. Deep learning
3. Parameters estimation
4. Supervised learning
5. More complex neural
6. Conclusion

Deep learning: overview

- Definition (attempt): set of learning methods that attempt to model data with complex architectures combining various non-linear transformations

$$\mathbf{x} \mapsto h(\mathbf{x}, \theta) \text{ such that } y \simeq h(\mathbf{x}, \theta)$$

- The basic building blocks of Deep Learning are **neural networks**.
- These bricks are combined to form **deep neural networks**.

Biological Neurons

images-deep-learning/fig10-1.jpg

Biological Neurons

images-deep-learning/fig10-2.jpg

Artificial Counterpart

McCulloch and Pitts (1943)

images-deep-learning/fig10-3.jpg

The Perceptron

Rosenblatt (1957), one of the simplest ANN architectures based on a *Linear Logit Unit* (LTU):

images-deep-learning/fig10-4.jpg

Common Step (Activation) Functions

- heaviside $= \begin{cases} 0 & \text{if } z < 0 \\ 1 & \text{if } z \geq 0 \end{cases}$
- sign $= \begin{cases} -1 & \text{if } z < 0 \\ 0 & \text{if } z = 0 \\ -1 & \text{if } z \geq 0 \end{cases}$
- How to train Threshold Logic Units (TLU)? By choosing weights w_1, w_2, w_3.
- When all the neurons in a layer are connected to every neuron in the previous layer, the layer is called a *fully connected layer* or a *dense layer*.
- All the inputs neurons x_i, $i = 1, 2, \ldots$ form the *input layer*, an extra bias feature ("bias neuron") is generally added ($x_0 = 1$) and output 1 all the time.

The Perceptron

Perceptron with two input neurons, one bias neuron, and three output neurons.

images-deep-learning/fig10-5.jpg

Artificial neuron

- *A neuron* is a non-linear application in its parameters that associates an output $h(\mathbf{X})$ to a matrix of input features $\mathbf{X}$:

$$h(\mathbf{X}) = \phi(\mathbf{X}\mathbf{w} + \mathbf{b})$$

- More precisely, the j-th neuron h_j is written

$$h_j(\mathbf{x}) = \phi(< w_j, \mathbf{x} > + b_j)$$

where

  - $< w_j, \mathbf{x} >= \sum_{r=1}^{p} w_j^r x^r$,
  - The quantities $w_j = (w_j^1, \ldots, w_j^p)$ are the weights of the input features $(x^1, \ldots, x^p)$,
  - b_j is the bias of neuron j,
  - ϕ is the **activation function**.

Basic model involving one neuron

We explain the output variable y by

$$y \simeq h(\mathbf{x}, \theta) = \phi(< w, \mathbf{x} > + b)$$

- If $\phi(z) = z$ and we only have one neurone $\Rightarrow$ classical linear function

Learning rule

Hebb's rule "Cells that fire together, wire together".

- Training of neurons: **connection weight between neurons is increased whenever they have the same output**
- For every output neuron that produced a wrong prediction
- The Perceptron learning rule reinforces the connection weights from the inputs that would have contributed to the correct prediction.
- Perceptron learning rule (weight update):

$$w_{i,j}^{\text{next step}} = w_{i,j} + \eta(y_j - \hat{y}_j)x_i,$$

- $w_{i,j}$ connection weight between ith input neuron and jth output neuron,
- x_i is ith input value,
- $\hat{y}_j$ output of jth output neuron,
- y_j target output,
- η learning rate.

Learning rule

- The decision boundary of each output neuron is linear, so Perceptrons are incapable of learning complex patterns (just like Logistic Regression classifiers).
- However, if learning instances are separable, this algorithm converges to a solution (*Perceptron convergence theorem*).
- Contrary to Logistic Regression classifiers, Perceptrons do not output a class probability, they make predictions based on hard thresholds.

Example: Iris Data

images-deep-learning/fig4-22.jpg

Example: Iris Data

- Load Iris dataset

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.linear_model import Perceptron

iris = load_iris()
print(iris.DESCR)
```

Example: Iris Data

- We will use petal length and petal width to classify a flower as Iris Setosa or "not Iris Setosa"

```
X = iris.data[:, (2, 3)]
# petal length, petal width
y = (iris.target == 0).astype(np.int)
# Iris Setosa?
```

- Plot petal length and width for Iris Setosa and "not Iris Setosa"
- Train perceptron with data and make a prediction for petal length 2 and petal width 0.5:

```
per_clf = Perceptron(max_iter=100,
  tol=-np.infty, random_state=42)
per_clf.fit(X, y)

y_pred = per_clf.predict([[2, 0.5]])
```

Classification problem

- Minsky and Papert (1969) showed shortcomings of simple Perceptrons (e.g., the *Exclusive OR* (XOR) classification problem cannot be implemented)

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

- These shortcomings can be eliminated using **Multi-Layer Perceptrons**.

Classification problem

images-deep-learning/fig10-6.jpg

Exercise: Resolving the XOR problem

Write a function "mlp_xor" which uses a Multi-Layer Perceptron with the weights on the previous slide to compute XOR.

Choice of the activation function ϕ

- If $Y \in \{0, 1\}$ and we want to predict $P(Y = 1|x)$: logistic function

$$\phi(z) = \frac{1}{1 + e^{-z}} \in [0, 1]$$

- *Generalization*: If $Y \in E$ (E finite space of cardinal K) and we want to predict $P(Y = e|x)$: *softmax*

$$\phi(z_1, \ldots, z_K) = \left(\frac{e^{z_j}}{\sum_{j=1\ldots K} e^{z_j}} \right)_{j=1\ldots K}$$

- Hyperbolic tangent function : $\phi = \tanh : \mathbb{R} \mapsto [-1, 1]$
- Threshold function : $\phi(z) = 1_{z>s} \in \{0, 1\}$
- Rectified linear unit (ReLU) $\phi(z) = max(z, 0)$
  - OK to predict positive values. Continuous but non differentiable

Plots

images-deep-learning/unnamed-chunk-1-

Various differentiability properties : important when we will have to optimize the w et b

Neural networks or multilayer perceptrons

- Structure composed of different hidden layers of neurons whose outputs serve as inputs to the neurons of the next layer
- Activation functions are the same in the different layers, only the last is different (adapted to the objective: classification or regression)

Example of neural network

images-deep-learning/mlp-network.png

Figure: Example of neural network

- Input layer: as many nodes as variables $x : p$
- Hidden layer: number of neurons, to be set but the user (here 4 then 3)
- Output layer: 1 node $= y$ for regression or binary classification and number of modalities for general classification

Neural network: formally

Let J_ℓ be the number of neurons in layer ℓ

- Layer 0 : $h^0(\mathbf{x}) = \mathbf{x} \in \mathbb{R}^p$
- For hidden layers $\ell = 1 \dots L$:
  - We create J_ℓ neurons : for every $j = 1 \dots J_\ell$:

$$\begin{aligned} a_j^{(\ell)}(\mathbf{x}) &= b_j^{(\ell)} + \sum_{m=1}^{J_{\ell-1}} W_{jm}^{(\ell)} h_m^{(\ell-1)}(\mathbf{x}) \\ &= b_j^{(\ell)} + < W_j^{(\ell)}, h^{(\ell-1)}(\mathbf{x}) > \\ h_j^{(\ell)}(\mathbf{x}) &= \phi(a_j^{(\ell)}(\mathbf{x})) \end{aligned}$$

- With vectors and matrices :

$$a^{(\ell)}(\mathbf{x}) = b^{(\ell)} + W^{(\ell)} h^{(\ell-1)}(\mathbf{x}) \in \mathbb{R}^{J_\ell}$$

$$h^{(\ell)}(\mathbf{x}) = \phi(a^{(\ell)}(\mathbf{x}))$$

where $W^{(\ell)}$ is matrix of size $J_\ell \times J_{\ell-1}$

Neural network: formally

- For the last layer $\ell = L + 1$:

$$a^{(L+1)}(\mathbf{x}) = b^{(L+1)} + W^{(L+1)} h^{(L)}(\mathbf{x}) \in \mathbb{R}^J$$
$$h^{(L+1)}(\mathbf{x}) = \psi(a^{(L+1)}(\mathbf{x}))$$

Neural network: finally

- $W^{(\ell)}$ is a weight matrix with J_ℓ rows and $J_{\ell-1}$ columns.
- $W^{(L+1)}$ is a weight matrix with 1 row and J_L columns $y \in \mathbb{R}$
- $$\mathbf{x} \mapsto h(\mathbf{x}, \theta) = \psi(a^{(L+1)}(\mathbf{x}))$$

  - If we are in a regression context $\psi(z) = z$,
  - If we are in a binary classification context ψ is the sigmoid function (prediction in $[0, 1]$).
  - If we are in a multiclass classification framework : $\psi = softmax$

Neural Network

- Basic architecture since each layer depends on the previous layer and not on the neurons of the same layer ($\Rightarrow$ recurrent neural networks)
- Parameters to tune or fit: - number of layers - number of neurons in each layer - hidden layer activation functions (ϕ) - Choice of the output activation function (ψ) driven by the dataset

Recurrent neural networks

- The output of a neuron can be the input of a neuron of the same layer.

Theoretical result

- Hornik (1991) proved that any smooth bounded function from $\mathbb{R}^p$ to $\mathbb{R}$ can be approximated by a one layer neural network with a finite number of neurons with the same activation function ϕ and $\psi = id$.
- Interesting result from a theoretical point of view.
- In practice :
  - Required number of neurons can be very large.
  - Strength of deep learning derives from the number of hidden layers

Outline

1. Introduction
2. Deep learning
3. Parameters estimation
4. Supervised learning
5. More complex neural
6. Conclusion

A quantity to minimize / maximize : loss function

- Parameters to estimate : $\theta =$ weights $W^{(\ell)}$ and bias $b_j^{(\ell)}$
- Classically : estimation by maximizing the (log)-likelihood
- Loss function: $= -\log$ likelihood
  - In the regression framework : $Y \sim \mathcal{N}(h(\mathbf{x}, \theta), I)$

$$\ell(h(\mathbf{x}, \theta), Y) = \|Y - h(\mathbf{x}, \theta)\|^2$$

  - For the binary classification $\{0, 1\}$: $Y \sim \mathcal{B}(1, h(\mathbf{x}, \theta))$

$$\ell(h(\mathbf{x}, \theta), Y) = -Y \log(h(\mathbf{x}, \theta)) - (1 - Y) \log(1 - h(\mathbf{x}, \theta))$$

    (cross-entropy)
  - For the multiclass classification

$$\ell(h(\mathbf{x}, \theta), Y) = -\sum_{e \in E} \mathbf{1}_{Y=e} \log p_\theta(Y = e|\mathbf{x})$$

Loss function: remark

- Ideally, we would like to minimize the classification error but it is not a differentiable function
- So we resort to the "cross-entropy" which can be differentiated

Penalized empirical risk

Risk

$$\mathbb{E}_Y[\ell(h(\mathbf{x}, \theta), Y)]$$

replaced by the empirical risk $\mathbb{E}_Y[\ell(h(\mathbf{x}, \theta), Y)]$

- For a training sample $(\mathbf{x}_i, Y_i)_{i=1\dots n}$

$$L_n(\theta) = \frac{1}{n} \sum_{i=1}^{n} \ell(h(\mathbf{x}_i, \theta), Y_i)$$

- Penalization :

$$L_n(\theta) = \frac{1}{n} \sum_{i=1}^{n} \ell(h(\mathbf{x}_i, \theta), Y_i) + \lambda \Omega(\theta)$$

with, for instance, $\Omega(\theta) = \sum_{\ell,i,j} (W_{ij}^{(\ell)})^2$ or $\Omega(\theta) = \sum_{\ell,i,j} |W_{ij}^{(\ell)}|$ to avoid overfitting.

Minimization by Stochastic gradient descent

Algorithm (by Rumelhart et al (1988))

- Choose an initial value of parameters θ and a learning rate η
- Repeat until a minimum is reached:
  - Split randomy the training set into N_B *batches* of size m ($n = m \times N_B$)
  - for each batch B set:

$$\theta := \theta - \eta \frac{1}{m} \sum_{i \in B} \nabla_\theta \left\{ \ell(h(\mathbf{x}_i, \theta), Y_i) + \lambda \Omega(\theta) \right\}$$

Remarks:

- Each iteration is called an *epoch*.
- The number of epochs is a parameter to tune
- Difficulty comes from the computation of the gradient

Calculus of the gradient for the regression

- $Y \in \mathbb{R}$.
- $R_i = \ell(h(\mathbf{x}_i, \theta), Y_i) = (Y_i - h(\mathbf{x}_i, \theta))^2$
- For any activation function ϕ (hidden layers) and ψ

Partial derivatives of R_i with respect to the weights of the last layer

- Derivatives of $R_i = (Y_i - h(\mathbf{x}_i, \theta))^2 = \left(Y_i - h^{(L+1)}(\mathbf{x}_i)\right)^2$ with respect to $(W_j^{(L+1)})_{j=1\dots J_L}$
- $a^{(L+1)}(\mathbf{x}) = b^{(L+1)} + W^{(L+1)} h^{(L)}(\mathbf{x}) \in \mathbb{R}^J$
- $$\begin{aligned} h(\mathbf{x}, \theta) &= h^{(L+1)}(\mathbf{x}) \\ &= \psi(a^{(L+1)}(\mathbf{x})) \\ &= \psi\left(b^{(L+1)} + \sum_{j=1}^{J_L} W_j^{(L+1)} h_j^{(L)}(\mathbf{x})\right) \end{aligned}$$
- $$\frac{\partial R_i}{\partial W_j^{(L+1)}} = -2\left(Y_i - h(\mathbf{x}_i, \theta)\right) \psi'\left(a^{(L+1)}(\mathbf{x}_i)\right) h_j^{(L)}(\mathbf{x}_i)$$

Partial derivatives of R_i with respect to the weights of the layer $L-1$

- Derivatives of $R_i = \left(Y_i - h^{(L+1)}(\mathbf{x}_i)\right)^2$ with respect to $(W_{jm}^{(L)})_{j=1\dots J_L, m=1\dots J_{L-1}}$

$$\frac{\partial R_i}{\partial W_{jm}^{(L)}} = -2\left(Y_i - h(\mathbf{x}_i, \theta)\right)\psi'\left(a^{(L+1)}(\mathbf{x}_i)\right)\frac{\partial}{\partial W_{jm}^{(L)}} a^{(L+1)}(\mathbf{x}_i)$$

Partial derivatives of R_i with respect to the weights of the layer $L-2$

$$
\begin{aligned}
a^{(L+1)}(\mathbf{x}) &= b^{(L+1)} + \sum_{j=1}^{J_L} W_j^{(L+1)} h_j^{(L)}(\mathbf{x}) \\
&= b^{(L+1)} + \sum_{j=1}^{J_L} W_j^{(L+1)} \phi\left(b_j^{(L)} + \sum_{m=1}^{J_{L-1}} W_{jm}^{(L)} h_m^{(L-1)}(\mathbf{x}) \right)
\end{aligned}
$$

$$
\begin{aligned}
\frac{\partial}{\partial W_{jm}^{(L)}} a^{(L+1)}(\mathbf{x}_i) &= W_j^{(L+1)} \phi'\left(b_j^{(L)} + \sum_{m=1}^{J_{L-1}} W_{jm}^{(L)} h_m^{(L-1)}(\mathbf{x}_i) \right) \\
&\quad \times h_m^{(L-1)}(\mathbf{x}_i) \\
&= W_j^{(L+1)} \phi'(a_j^L(\mathbf{x}_i)) h_m^{(L-1)}(\mathbf{x}_i)
\end{aligned}
$$

Forward-Backward algorithm (at each iteration)

After some light effort, recurrence formula

- Given the current parameters
  - **Forward step**: From layer 1 to layer $L+1$, compute the $a_j^\ell(\mathbf{x}_i), \phi(a_j^\ell(\mathbf{x}_i))$
  - **Backward step**: From layer $L+1$ to layer 1, compute the partial derivatives (recurrence formula update)

Tuning the algorithm

- η: learning rate of the gradient descent
  - if η too small, really slow convergence with possibly reaching of a local minimum
  - if η too large, maybe oscilliation around an optimum without stabilisation
  - Adaptive choice of η (decreasing η)
- Batch calculation reduces the number of quantities to be stored in the forward / backward

Obviously

Many improved versions of the maximisation algorithm (momentum correction, Nesterov accelerated gradient, etc. . .)

Automatic differentiation

Success of the neural network comes from automatic differentiation, i.e. automatisation of the previously described forward-backward procedure to compute the derivatives : `Tensorflow`

Deep neural networks

- deep neural networks: two or more hidden layer
- Rumelhart et al (1986): backpropagation training algorithm:
  1. forward pass (make prediction)
  2. measure output error
  3. compute how much each neuron in last hidden layer contributed to the error
  4. measure how much each neuron in previous hidden layer contributed to error in subsequent hidden layer
  5. slightly tweak connection weights to reduce error

Deep neural networks

- Rumelhart replaced stepwise function with a logistic function as activation function $\sigma(z) = 1/(1 + \exp(-z))$
- alternative activation functions
  - hyperbolic tangent function $\tanh(z) = 2\sigma(2z) - 1$
  - rectified linear unit (ReLU) function $\max(0, z)$

Activation functions

images-deep-learning/fig10-8.jpg

Modern Multi-Layer Perceptron

images-deep-learning/fig10-9.jpg

Modern Multi-Layer Perceptron

- typical usage: divide data in
  - training set
  - validation set
  - test set
- nowadays mostly ReLU for hidden layers
- output layer:
  - softmax if classification tasks are mutually exclusive
  - if not mutually exclusive logistic function
  - for regression tasks: no activation function for output layer

Universal Approximation Theorem

- universal approximation theorem: one single hidden layer with sufficiently many neurons can approximate any continuous function
- however, for practical purposes, multiple hidden layers are needed
- with a single hidden layer, the number of required neurons may grow exponentially with complexity
- multiple hidden layers avoid this problem (see next slide)

Multiple Layers

images-deep-learning/mlp-fig6-5.jpg

Multiple Layers

- modern deep neural networks have often ca. 10 hidden layers
- this causes difficulties:
  - vanishing gradients problem (or the related exploding gradients problem)
  - training slow
  - model with a million parameter leads to risk of overtraining

Vanishing Gradients Problem

Glorot and Bengio (2010, "Understanding the Difficulty of Training Deep Feedforward Neural Networks")

images-deep-learning/fig11-1.jpg

Vanishing Gradients Problem

- solution by Glorot and Bengio:
  - variance of the outputs in each layer has to be equal to the variance of inputs to ensure signal flow
  - gradients need to have equal variance before and after flowing through a layer in the reverse direction
- practical solution: initialize connection weights randomly to achieve these goals, for normal distribution $\sigma = \sqrt{2/(n_{\text{inputs}} + n_{\text{outputs}})}$ ("Xavier initialization")
- further solutions: use ReLU rather than sigmoid activation function
  - ReLU does not saturate for positive values
  - but ReLU has the "dying ReLU problem": during training, some neurons output only 0s
- $\rightarrow$ leaky ReLU $\max(\alpha z, z)$

Leaky ReLU activation function

images-deep-learning/fig11-2.jpg

ELU activation function

images-deep-learning/fig11-3.jpg

Batch normalization

- Ioffe and Szegedy (2015): Batch Normalization
- makes sure that the vanishing/exploding gradients problem does not come back during training
- two new parameters per layer: add a scaling and a shifting before the activation function

Transfer Learning

images-deep-learning/fig11-4.jpg

Transfer Learning

- freezing lower layers
- model zoos

Optimization

- fast optimizer for backpropagation are important
- there has been a lot of research on optimizers
- traditionally gradient descent algorithms
- newer: momentum optimizers (faster)
- Nesterov momentum optimization (even faster)
- adaptive learning rate algorithms (even faster than Nesterov):
  - AdaGrad
  - RMSProp
  - Adam Optimization
- however, new research suggests that adaptive optimization performs poorly on some datasets (Wilson, 2017)
- Hessian based algorithms don't work here since n^2 Hessians would be needed per output (n being the number of parameters)

Avoiding Overfitting

- Deep Neural Networks: tens of thousands of parameters, sometimes even millions
- Overfitting can be problem
- Several methods to avoid overfitting:
  - Reduce the capacity of the network.
  - Dropout
  - Weight Regularization
  - Data Augmentation (get more training data)

Early Stopping

- Interrupt training when its performance on the validation set starts dropping

Dropout

- Dropout is one of the most effective and most commonly used regularization techniques for neural networks (Hinton (2012) and further detailed by Srivastava et al.)
- At every training step (each layer), every neuron has a probability of p of being temporarily "dropped out" (ignore during training step)

Dropout

Let's say a given layer would normally return a vector [0.2, 0.5, 1.3, 0.8, 1.1] for a given input sample during training. After applying dropout, this vector will have a few zero entries distributed at random: for example, [0, 0.5, 1.3, 0, 1.1].

- The dropout rate p is the fraction of the features that are zeroed out; it's usually set between 20% and 50%
- For predictions (test time), all neurons are used, i.e., no units are dropped out; instead, the layer's output values are scaled down by a factor equal to the dropout rate, to balance for the fact that more units are active than at training time.

Dropout

images-deep-learning/fig11-9.jpg

Dropout

- Neural network cannot rely to much on just a few neurons.
- Neurons become less sensitive to slight changes of inputs.
- Another way to view this: in every step there is a slightly different neural network.
- For prediction an average of the neural networks is used.

Weight Regularization

- Simpler models are less likely to overfit than complex ones.
- A simple model is a model where the distribution of parameter values has less entropy (or a model with fewer parameters).
- A common way to mitigate overfitting is to put constraints on the complexity of a network by forcing its weights to take only small values, which makes the distribution of weight values more regular.
- Weight regularization is done by adding to the loss function of the network a cost associated with having large weights.

Weight Regularization

Max-Norm Regularization

L2 regularization: The cost added is proportional to the square of the value of the weight coefficients (the L2 norm of the weights). L2 regularization is also called weight decay in the context of neural networks.

- Constraint weight for each neuron to be $||w||_2 \leq r$ for some r, where $||w||_2$ is the L_2 norm of w
- Reducing r reduces overfitting

A *regularizer_l2(0.001)* means every coefficient in the weight matrix of the layer will add $0.001 * weight_coefficient_value$ to the total loss of the network. Note that because this penalty is only added at training time, the loss for this network will be much higher at training time than at test time.

Data Augmentation

- generate new training instances from existing ones
- realistic training instances

Data Augmentation

images-deep-learning/fig11-10.jpg

Fine-Tuning Neural Network Hyperparameters

Even in a simple MLP you can change:

- The number of layers,
- The number of neurons per layer,
- The type of activation function to use in each layer,
- The weight initialization logic...

How do you know what combination of hyperparameters is the best for your task?

Fine-Tuning Neural Network Hyperparameters

Try many combinations of hyperparameters and see which one works best on the validation set. You can explore the hyperparameter space with

- 'GridSearchCV',
- 'RandomizedSearchCV'.

Typical regression architecture

Hyperparameter	Typical value
# input neurons	One per input feature (e.g., 28 x 28 = 784 for MNITS)
# hidden layers	Depends on the problem, but tipically 1 to 5
# neurons per hidden layer	Depends on the problem, but typically 10 to 100
# output neurons	1 per prediction dimension
Hidden activation	ReLU (or SELU)
Output activation	None, or ReLU/softplus (if positive outputs), or logistic/tanh (if bounded outputs)
Loss function	MSE or MAE/Huber (if outliers)

Typical classification architecture

Hyperparameter	Binary classification	Multilabel binary classification	Multiclass classification
input and hidden layer	Same as regression	Same as regression	Same as regression
# output neurons	1	1 per label	1 per class
Output layer activation	Logistic	Logistic	Softmax
Loss function	Cross entropy	Cross entropy	Cross entropy

Outline

1. Introduction
2. Deep learning
3. Parameters estimation
4. Supervised learning
5. More complex neural
6. Conclusion

Example: Handwritten Digits Recognition

images-deep-learning/fig3-1.jpg

Example: Handwritten Digits Recognition

images-deep-learning/mnist_details.png

Example: Handwritten Digits Recognition

- The gray levels are stored in a vector $\mathbf{x}_i = (x_i^r)_{r=1\ldots p}$, $p = 784$
- $y_i \in \{0, \ldots, 9\}$: image label
  - y_i : observed / known for a learning sample
  - y must be predicted for a new image $\mathbf{x}$

Formally

- We consider n objects (images, texts, sound ...), described by p characteristics.
- For each object i, these characteristics are stored in a vector $\mathbf{x}_i = (x_i^1, \dots, x_i^p)$ of $\mathbb{R}^p$.
- An output variable y_i is each object i is assigned
  - If $y_i \in \mathbb{R}^d$: we talk about regression
  - If $y_i \in E$ with E set, we talk about - discriminating if $E = \{0, 1\}$; - classification if $E = \{0, \dots, 9\}$ for example - shape recognition if $E = \{\text{dog}, \text{groundhog}, \dots\}$
- **Goal**: predict the output y for a new set of characteristics $\mathbf{x}$
- **How do we do it?**: learn (on a set of learning data = training) a prediction rule or classification and provide this rule to apply it to $\mathbf{x}$

Outline

1. Introduction
2. Deep learning
3. Parameters estimation
4. Supervised learning
5. More complex neural
6. Conclusion

Main types of neural networks

Main Types of Neural Networks:

- Recurrent Neural Network (RNN)
- Long / Short Term Memory (LSTM)
- Auto Encoder (AE)
- Markov Chain (MC)
- Convolutional Neural Network (CNN)
- Generative Adversarial Network (GAN)

Convolutional neural network (CNN)

- LeNet by LeCun et al., 1998
- When we transform an image into a vector : loss of spatial coherence of the image (shapes, . . .)
- CNN revolutionized Image Analysis (LeCun)
- Composed of different convolution layers, pooling layers and fully connected layers.

In one picture

images-deep-learning/Cnn_layer_standf

Figure: Architecture of a convolutive neural network (Stanford credit)

Convolutional neural network (CNN)

Currently the dominant approach for neural networks:

- Use many different copies of the same feature detector with different positions.
  - Could also replicate across scale and orientation (tricky and expensive)
  - Replication greatly reduces the number of free parameters to be learned.
- Use several different feature types, each with its own map of replicated detectors.
  - Allows each patch of image to be represented in several ways.

images-deep-learning/Plot_CNN_weight

Figure: The red connections all have the same weight.

Le Net

Yann LeCun and his collaborators developed a really good recognizer for handwritten digits by using backpropagation in a feedforward net with:

- Many hidden layers
- Many maps of replicated units in each layer.
- Pooling of the outputs of nearby replicated units.
- A wide net that can cope with several characters at once even if they overlap.
- A clever way of training a complete system, not just a recognizer.

This net was used for reading 10% of the checks in North America.
Look the impressive demos of LENET

A neural network for ImageNet

Alex Krizhevsky (NIPS 2012) developed a very deep convolutional neural net of the type pioneered by Yann Le Cun. Its architecture was:

- 7 hidden layers not counting some max pooling layers.
- The early layers were convolutional.
- The last two layers were globally connected.

The activation functions were:

- Rectified linear units in every hidden layer. These train much faster and are more expressive than logistic units.
- Competitive normalization to suppress hidden activities when nearby units have stronger activities. This helps with variations in intensity.

A neural network for ImageNet

Train on random 224x224 patches from the 256x256 images to get more data. Also use left-right reflections of the images.

- At test time, combine the opinions from ten different patches: The four 224x224 corner patches plus the central 224x224 patch plus the reflections of those five patches.

Use "dropout" to regularize the weights in the globally connected layers (which contain most of the parameters).

- Dropout means that half of the hidden units in a layer are randomly removed for each training example.
- This stops hidden units from relying too much on other hidden units.

A neural network for ImageNet

images-deep-learning/ImageNet_NeuralNetwork.png

Convolution layer

- Image $\mathcal{I}(u, v, c)$: each pixelwith (u, v) is described by C color levels (channels), par exemple RGB (red, green, blue) $\Rightarrow$: array of size (M, N, C)
- Each neuron relies on a linear combination of the signals of a small region of the image:

$$K_{u,v} * \mathcal{I}(c) = \sum_{n=-k}^{k} \sum_{m=-k}^{k} K_l(n, m, c)\mathcal{I}(u + m, v + n, c)$$

$$h_{u,v} = \phi(K_{u,v} * \mathcal{I}(c) + b_{u,v})$$

- We obtain a new neuron by moving this window - The amount by which the filter shifts is the **stride** - if we don't move a lot : redundancy of information - if we move a lot: loss of information

Pooling layer (subsampling)

- Averaging or computing maximum on small areas
- ϕ can be applied before or after pooling
- Allows to reduce the dimension (number of variables to be treated) but also to make the network less sensitive to possible translations of the image

Finally

- After several layers and convolution / pooling, we apply one or more layers of *fully connected layers* (= network shown before)

Towards increasingly complex network architectures

- Networks now much more complex
- Can be processed with Graphical Processor Unit (GPU) cards
- Results of the *Large Scale Visual Recognition Challenge (ILSVRC)*

images-deep-learning/depth_error.png

Figure: Error rate and depths of the network

Outline

1. Introduction
2. Deep learning
3. Parameters estimation
4. Supervised learning
5. More complex neural
6. Conclusion

Conclusion

- Ultra-fast vision of the definition of a deep neural network
- In practice: expertise is based on how to combine layers and different types of neurons
- Additional References:
  - Wikipedia - Wikistat course by Philippe Besse (Toulouse)
  - Deep Learning by Yoshua Bengio
    `images-deep-learning/livre_bengio.jpg`
  - Video Collège de France
  - Results and conferences by Francis Bach